



Industrial Fabric Quality Inspection using Support Vector Machine (SVM)

Problem Statement

In industrial manufacturing, maintaining consistent fabric quality is critical for reducing defects and improving production efficiency. This project aims to classify fabric quality using a Support Vector Machine (SVM) by analyzing inspection parameters such as thread count, GSM, tensile strength, shrinkage, machine conditions, and environmental factors. The model helps automate quality inspection and supports accurate decision-making in textile industries.

In []:

Steps to be followed

1. Importing Required Libraries
2. Load Dataset
3. Data Preprocessing
4. Checking Null values
5. Data Wrangling (Filling Missing values)
6. Data Visualization
7. Label Encoding
8. Feature Selection (x,y)
9. Train-Test Split
10. Feature Scaling
11. SVM Model Prediction
12. Model Evaluation (Accuracy Score, Classification Report)
13. SVM Kernel Comparison and Performance Evaluation
14. Verifying Target Class Labels
15. Confusion Matrix
16. Cross Validation Score
17. Multi-Class ROC Curve and AUC Analysis for SVM
18. GridSearchCV and RandomizedSearchCV
19. Comparison of Hyperparameter Tuning Methods (GridSearch vs RandomSearch)
20. Improving Model Performance with Advanced Algorithms

Importing Required Libraries

```
In [1]: import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px
import math
```

```
In [2]: from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedS
from sklearn.metrics import accuracy_score, precision_score, classification_rep
from sklearn.preprocessing import OneHotEncoder, LabelEncoder, StandardScaler,
from sklearn.svm import SVC
```

```
In [52]: import warnings
warnings.filterwarnings("ignore")
```

Loading the Dataset

```
In [3]: df=pd.read_csv(r"C:\Users\HP\Downloads\Industrial Fabric Quality Inspection.csv")
df
```

```
Out[3]:
```

	thread_count	gsm	tensile_strength	shrinkage_percent	color_fastness
0	166.0	383.0	NaN	7.744704	1
1	273.0	107.0	106.508683	1.191272	1
2	274.0	352.0	97.100410	NaN	3
3	NaN	200.0	17.876660	4.001214	1
4	115.0	348.0	62.746013	2.529863	2
...	...	...	...	...	...
25745	126.0	91.0	67.381671	7.229657	4
25746	327.0	NaN	23.432368	5.120096	2
25747	80.0	NaN	NaN	8.487796	3
25748	298.0	375.0	49.096078	7.382152	1
25749	285.0	341.0	102.001693	9.140029	2

25750 rows × 23 columns

Data Preprocessing

```
In [4]: df.shape
```

```
Out[4]: (25750, 23)
```

```
In [5]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 25750 entries, 0 to 25749
Data columns (total 23 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   thread_count                          24460 non-null  float64
1   gsm                                   24465 non-null  float64
2   tensile_strength                     24458 non-null  float64
3   shrinkage_percent                   24467 non-null  float64
4   color_fastness                      25750 non-null  int64
5   fabric_thickness                    25750 non-null  float64
6   defect_count                        25750 non-null  int64
7   elongation_percent                  25750 non-null  float64
8   moisture_absorption                 25750 non-null  float64
9   fabric_type                          24458 non-null  object
10  weave_type                          24463 non-null  object
11  finish_type                         25750 non-null  object
12  production_method                   25750 non-null  object
13  batch_id                           25750 non-null  int64
14  roll_number                         25750 non-null  int64
15  inspection_time_minutes              25750 non-null  float64
16  warehouse_id                        25750 non-null  object
17  operator_name                       25750 non-null  object
18  inspection_shift                     24448 non-null  object
19  machine_temperature                 25750 non-null  float64
20  humidity_level                      25750 non-null  float64
21  inspection_notes                     25750 non-null  object
22  fabric_quality                      25750 non-null  object
dtypes: float64(10), int64(4), object(9)
memory usage: 4.5+ MB
```

```
In [6]: df.describe()
```

```
Out[6]:
```

	thread_count	gsm	tensile_strength	shrinkage_percent	color_fa
count	24460.000000	24465.000000	24458.000000	24467.000000	25750.
mean	213.498814	243.915512	67.523555	5.245481	2.
std	77.882638	89.561967	30.300540	2.739544	1.
min	80.000000	90.000000	15.000885	0.500959	1.
25%	146.000000	166.000000	41.367865	2.858116	2.
50%	213.000000	244.000000	67.690981	5.266917	3.
75%	281.000000	322.000000	93.936922	7.627155	4.
max	349.000000	399.000000	119.995480	9.999924	5.

The dataset contains both numerical and categorical variables related to fabric production and inspection.

```
In [ ]:
```

Checking Null Values

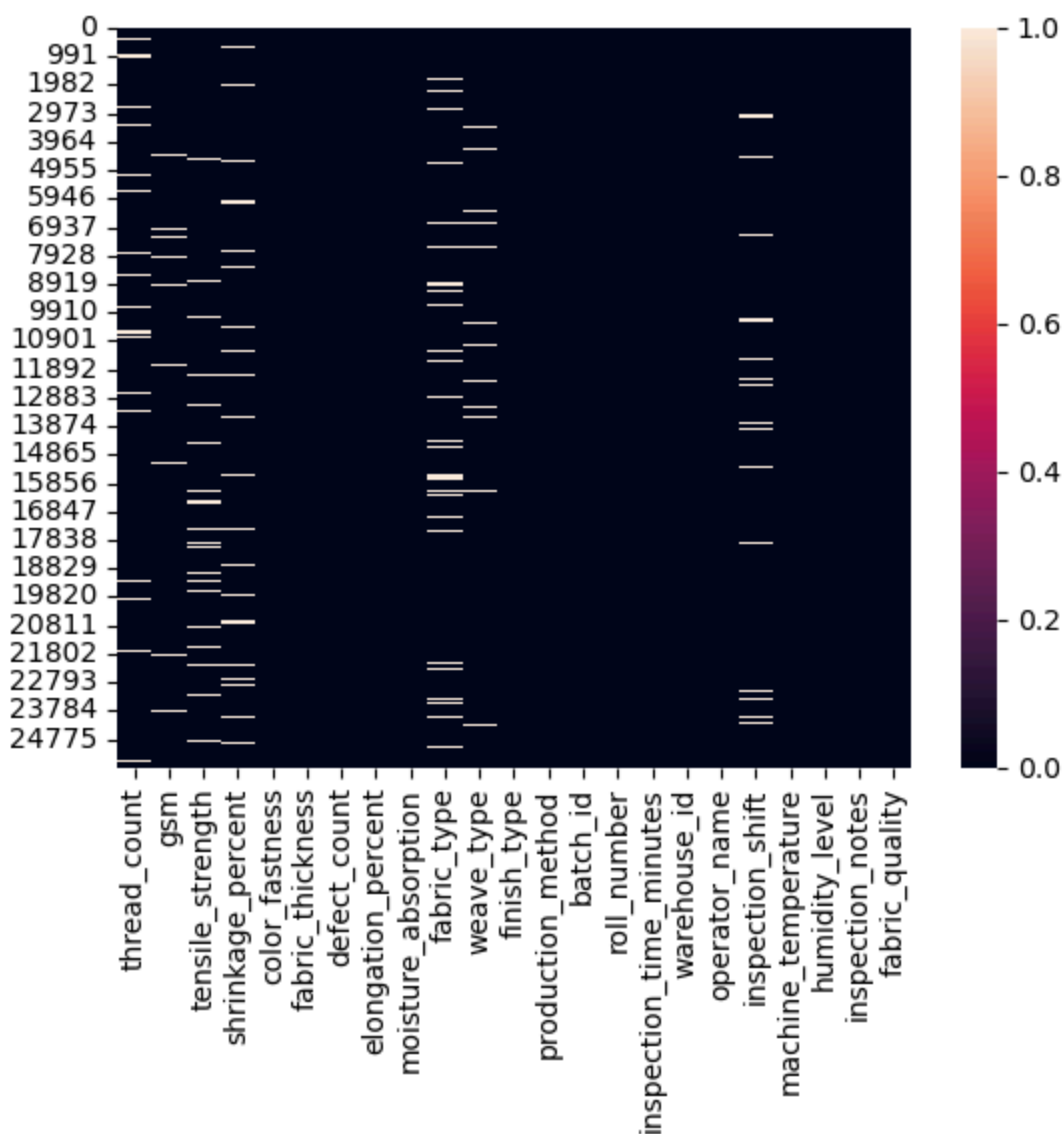
Missing values are identified to ensure data quality before training the model.

```
In [7]: df.isnull().sum()
```

```
Out[7]: thread_count      1290
        gsm                1285
        tensile_strength   1292
        shrinkage_percent  1283
        color_fastness     0
        fabric_thickness   0
        defect_count       0
        elongation_percent  0
        moisture_absorption 0
        fabric_type        1292
        weave_type         1287
        finish_type        0
        production_method  0
        batch_id           0
        roll_number        0
        inspection_time_minutes 0
        warehouse_id       0
        operator_name      0
        inspection_shift   1302
        machine_temperature 0
        humidity_level     0
        inspection_notes   0
        fabric_quality     0
        dtype: int64
```

```
In [8]: sns.heatmap(df.isnull())
```

```
Out[8]: <Axes: >
```



```
In [9]: df.columns
```

```
Out[9]: Index(['thread_count', 'gsm', 'tensile_strength', 'shrinkage_percent',
               'color_fastness', 'fabric_thickness', 'defect_count',
               'elongation_percent', 'moisture_absorption', 'fabric_type',
               'weave_type', 'finish_type', 'production_method', 'batch_id',
               'roll_number', 'inspection_time_minutes', 'warehouse_id',
               'operator_name', 'inspection_shift', 'machine_temperature',
               'humidity_level', 'inspection_notes', 'fabric_quality'],
              dtype='object')
```

Data Wrangling (Filling Null/Missing Values)

Some numerical features contain missing values which will be handled using

median imputation.

```
In [10]: num_cols=['thread_count', 'gsm', 'tensile_strength', 'shrinkage_percent']  
df[num_cols] = df[num_cols].fillna(df[num_cols].median())
```

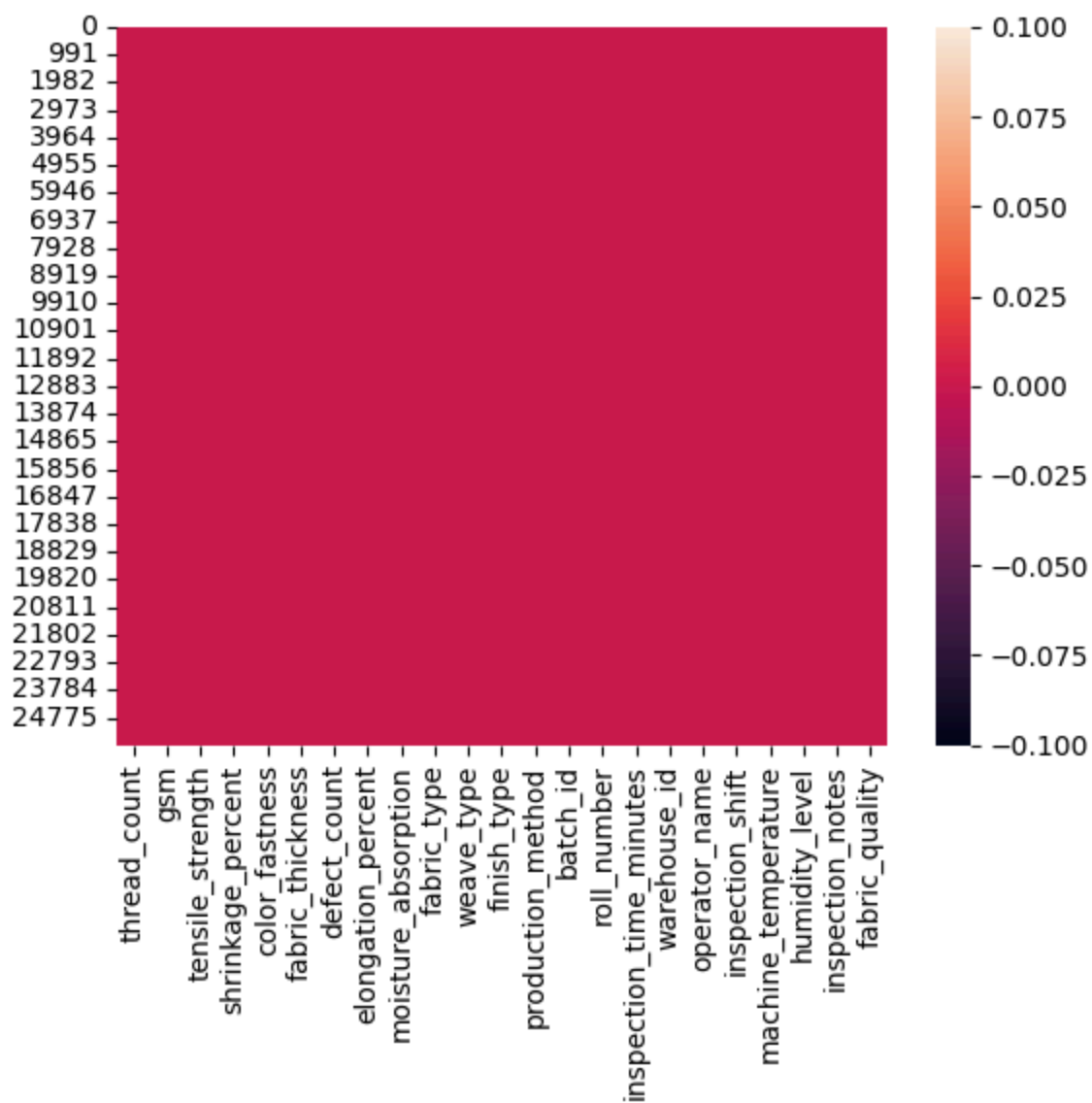
```
In [11]: cat_cols=['fabric_type', 'weave_type', 'inspection_shift']  
df[cat_cols] = df[cat_cols].fillna(df[cat_cols].mode().iloc[0])
```

```
In [12]: df.isnull().sum()
```

```
Out[12]: thread_count      0  
gsm                      0  
tensile_strength         0  
shrinkage_percent        0  
color_fastness           0  
fabric_thickness         0  
defect_count             0  
elongation_percent       0  
moisture_absorption      0  
fabric_type              0  
weave_type               0  
finish_type              0  
production_method        0  
batch_id                 0  
roll_number              0  
inspection_time_minutes  0  
warehouse_id             0  
operator_name            0  
inspection_shift         0  
machine_temperature      0  
humidity_level           0  
inspection_notes         0  
fabric_quality           0  
dtype: int64
```

```
In [13]: sns.heatmap(df.isnull())
```

```
Out[13]: <Axes: >
```



```
In [14]: df.dtypes
```



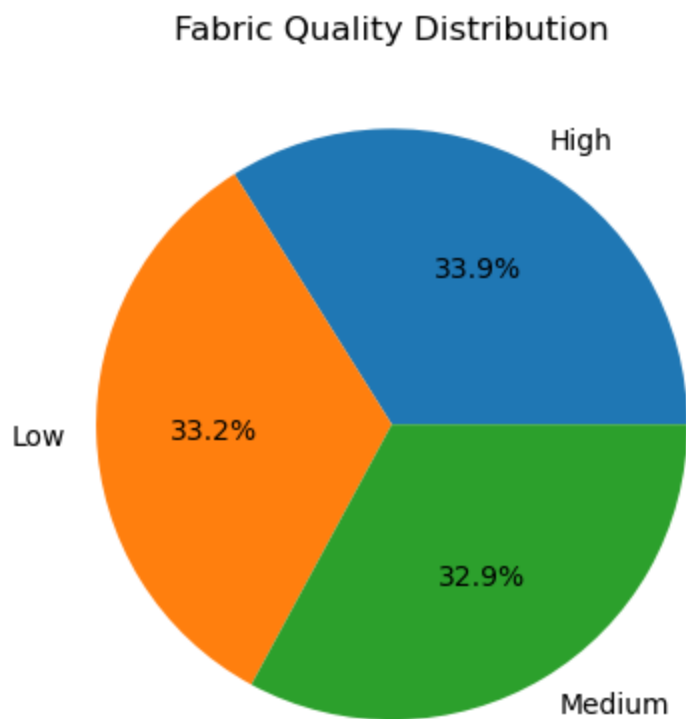
```
Out[14]: thread_count      float64
gsm                        float64
tensile_strength          float64
shrinkage_percent         float64
color_fastness            int64
fabric_thickness          float64
defect_count              int64
elongation_percent        float64
moisture_absorption        float64
fabric_type               object
weave_type                object
finish_type               object
production_method          object
batch_id                  int64
roll_number               int64
inspection_time_minutes    float64
warehouse_id              object
operator_name              object
inspection_shift           object
machine_temperature        float64
humidity_level             float64
inspection_notes           object
fabric_quality             object
dtype: object
```

```
In [15]: df.columns
```

```
Out[15]: Index(['thread_count', 'gsm', 'tensile_strength', 'shrinkage_percent',
'color_fastness', 'fabric_thickness', 'defect_count',
'elongation_percent', 'moisture_absorption', 'fabric_type',
'weave_type', 'finish_type', 'production_method', 'batch_id',
'roll_number', 'inspection_time_minutes', 'warehouse_id',
'operator_name', 'inspection_shift', 'machine_temperature',
'humidity_level', 'inspection_notes', 'fabric_quality'],
dtype='object')
```

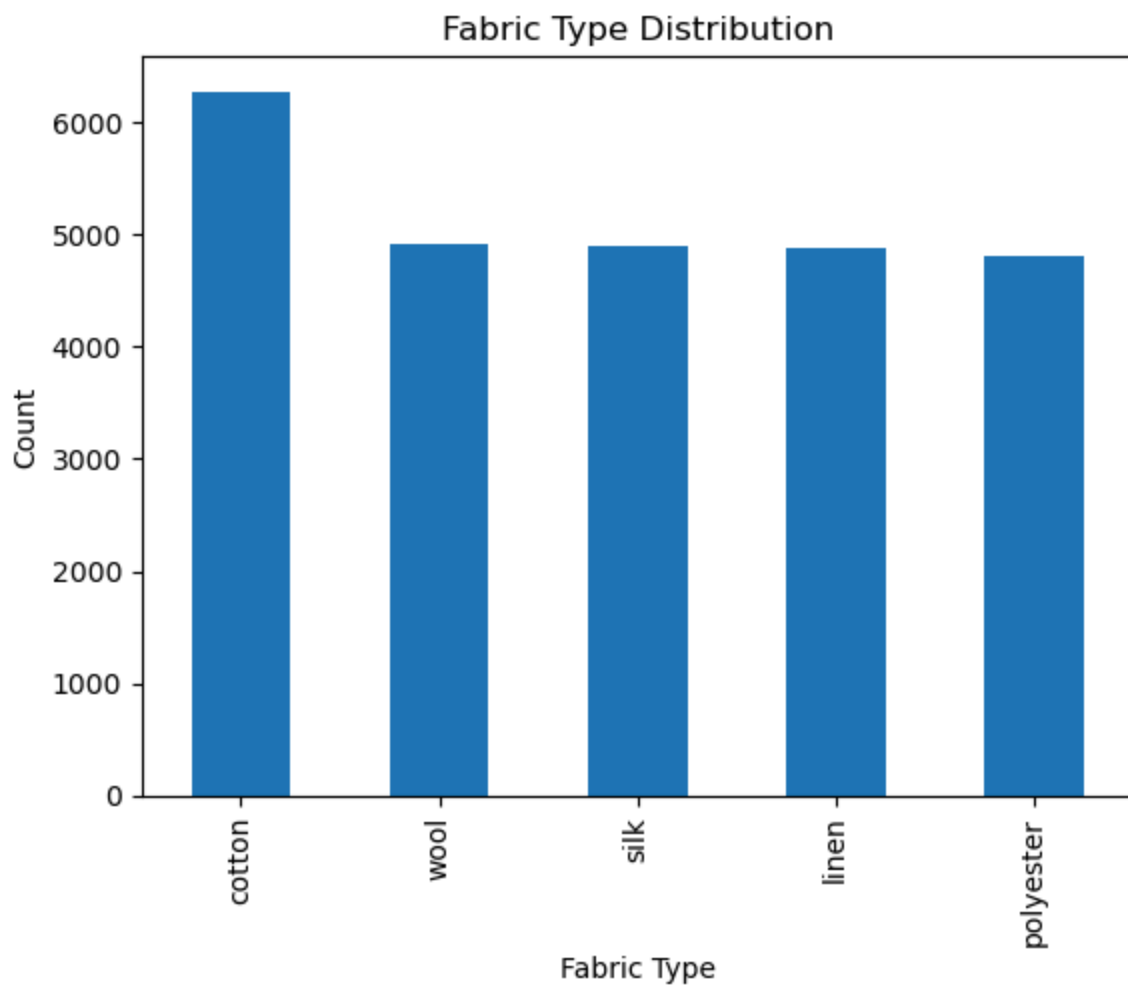
Data Visualization

```
In [16]: plt.figure()
df["fabric_quality"].value_counts().plot(
    kind="pie", autopct="%1.1f%%"
)
plt.title("Fabric Quality Distribution")
plt.ylabel("")
plt.show()
```



The dataset contains a balanced distribution of fabric quality classes, which helps improve classification performance.

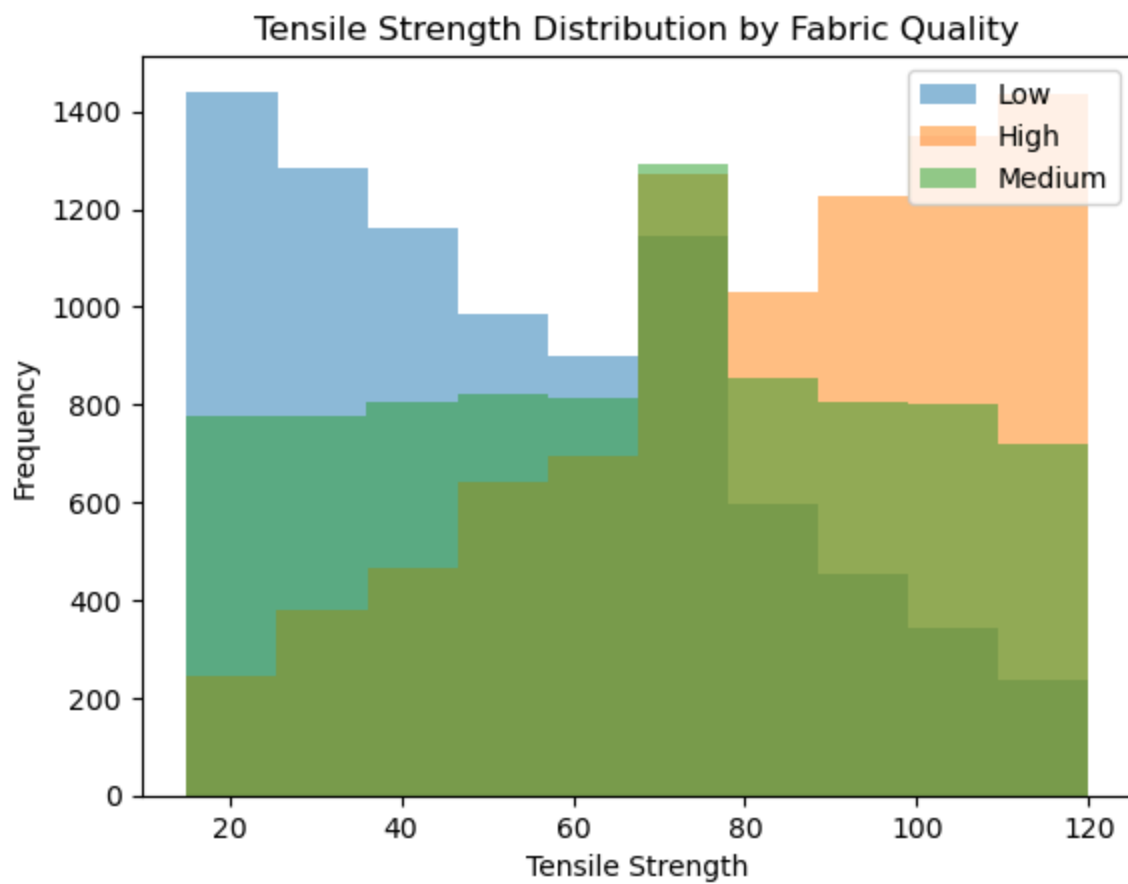
```
In [17]: plt.figure()
df["fabric_type"].value_counts().plot(kind="bar")
plt.title("Fabric Type Distribution")
plt.xlabel("Fabric Type")
plt.ylabel("Count")
plt.show()
```



Cotton fabrics are most common, while other fabric types appear in relatively similar proportions.

```
In [18]: plt.figure()
for q in df["fabric_quality"].unique():
    plt.hist(
        df[df["fabric_quality"] == q]["tensile_strength"].dropna(),
        alpha=0.5,
        label=q
    )

plt.title("Tensile Strength Distribution by Fabric Quality")
plt.xlabel("Tensile Strength")
plt.ylabel("Frequency")
plt.legend()
plt.show()
```



Higher tensile strength is strongly associated with higher fabric quality.

In []:

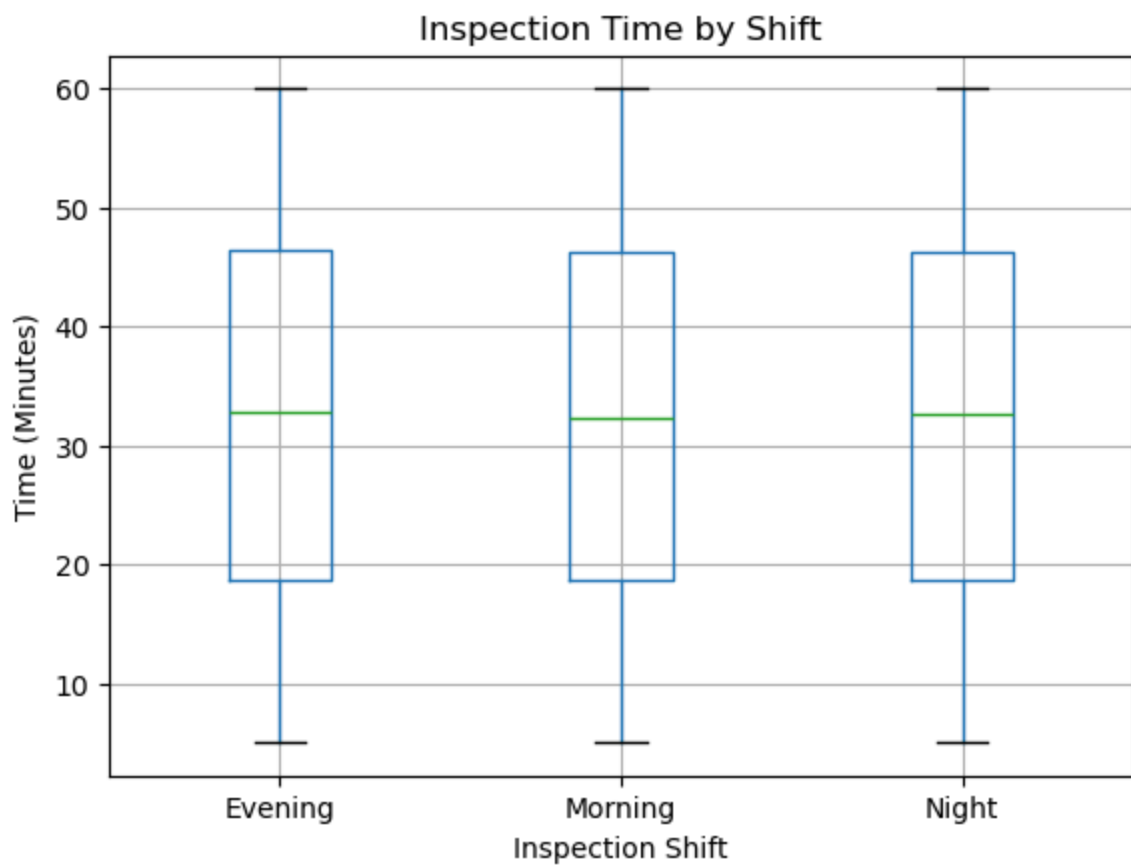
```
In [19]: fig = px.box(  
    df,  
    x="fabric_quality",  
    y="shrinkage_percent",  
    title="Shrinkage Percentage by Fabric Quality"  
)  
fig.show()
```

Lower shrinkage is an indicator of better fabric quality.

In []:

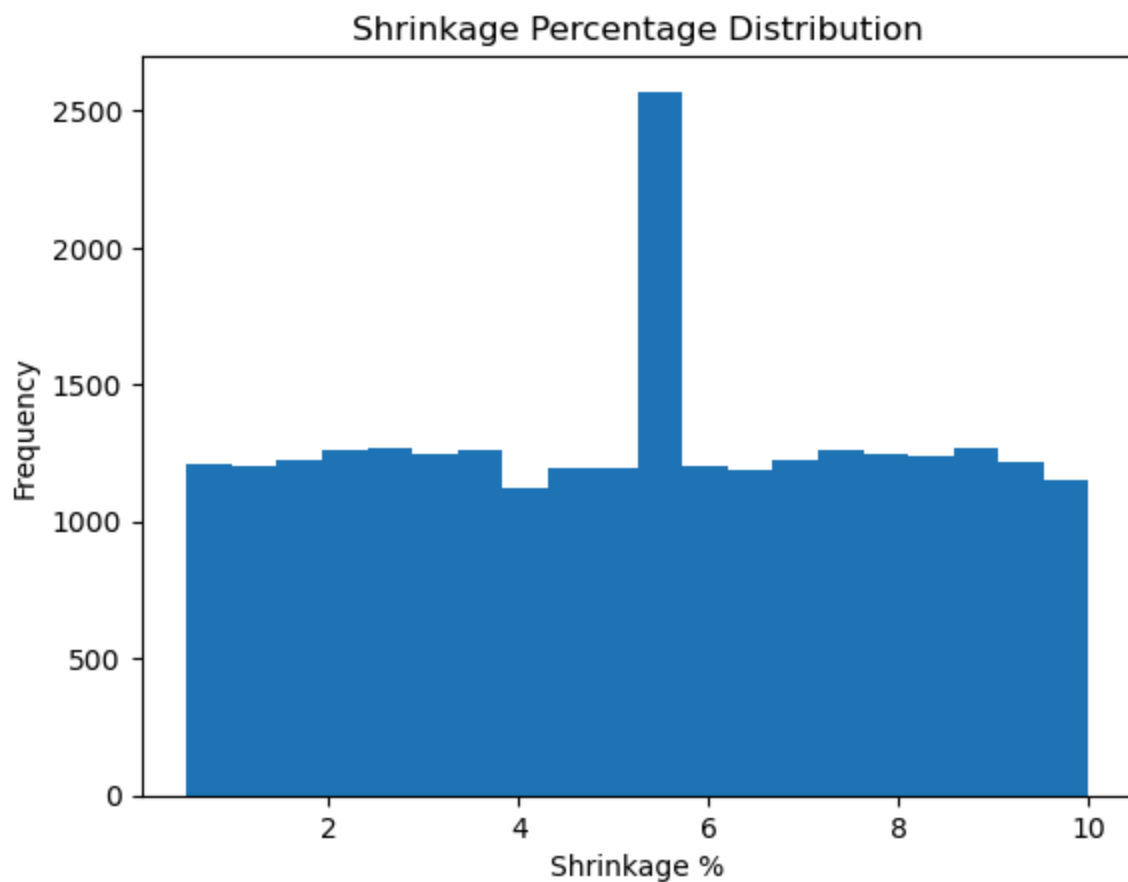
```
In [20]: plt.figure()
df.boxplot(
    column="inspection_time_minutes",
    by="inspection_shift"
)
plt.title("Inspection Time by Shift")
plt.suptitle("")
plt.xlabel("Inspection Shift")
plt.ylabel("Time (Minutes)")
plt.show()
```

<Figure size 640x480 with 0 Axes>



Inspection time remains consistent across all shifts, indicating standardized inspection procedures.

```
In [21]: plt.figure()
plt.hist(df["shrinkage_percent"].dropna(), bins=20)
plt.title("Shrinkage Percentage Distribution")
plt.xlabel("Shrinkage %")
plt.ylabel("Frequency")
plt.show()
```



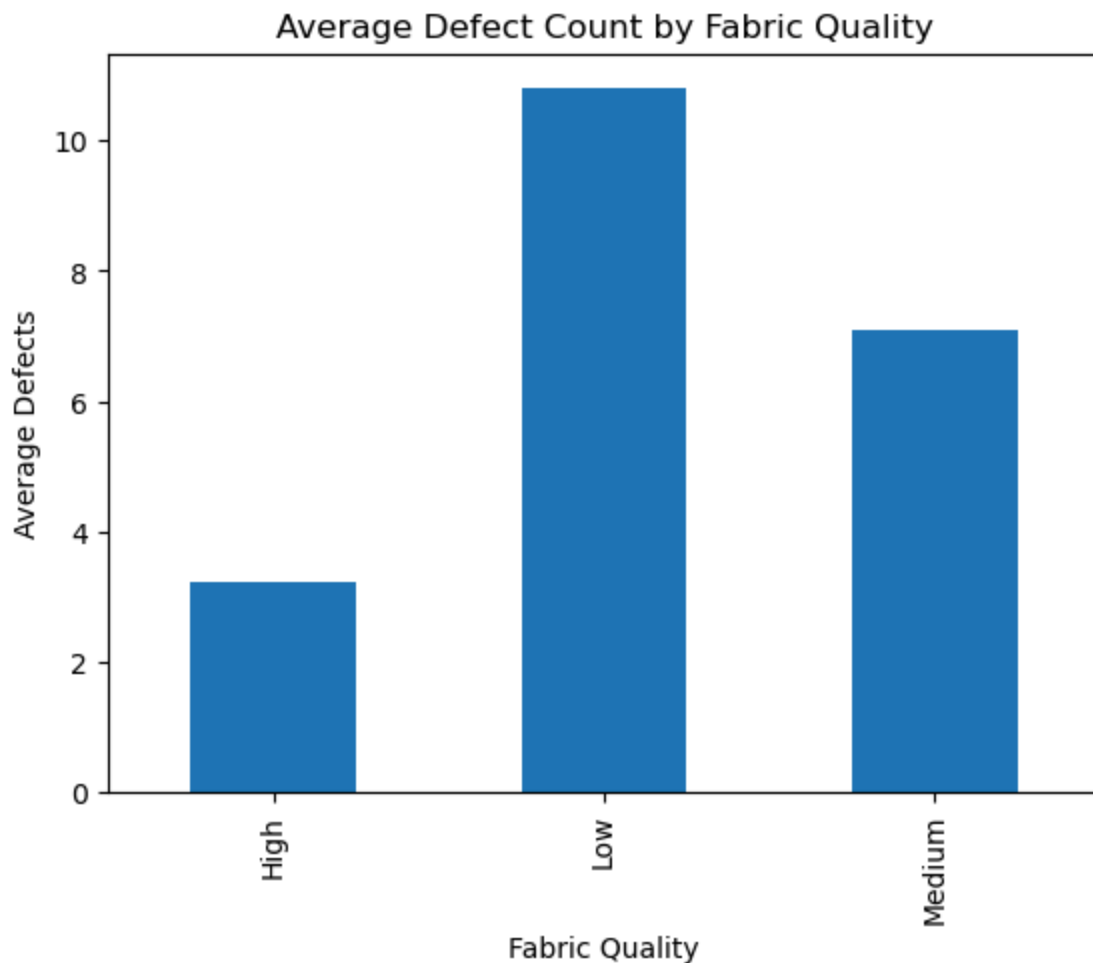
Fabric shrinkage is generally moderate, with most values concentrated in the mid-range.

```
In [22]: warehouse_quality = (  
    df.groupby(["warehouse_id", "fabric_quality"])  
    .size()  
    .reset_index(name="count")  
)  
  
fig = px.bar(  
    warehouse_quality,  
    x="warehouse_id",  
    y="count",  
    color="fabric_quality",  
    barmode="stack",  
    title="Fabric Quality Distribution by Warehouse"  
)  
fig.show()
```

Fabric quality distribution is consistent across warehouses, suggesting similar production standards.

```
In [23]: quality_defects = df.groupby("fabric_quality")["defect_count"].mean()

plt.figure()
quality_defects.plot(kind="bar")
plt.title("Average Defect Count by Fabric Quality")
plt.xlabel("Fabric Quality")
plt.ylabel("Average Defects")
plt.show()
```

Higher defect counts are associated with lower fabric quality.

Label Encoding

Categorical features are converted into numerical format using label encoding.

```
In [24]: label = LabelEncoder()
cat_col = ['fabric_type', 'weave_type', 'finish_type', 'production_method', 'warehouse',
           'operator_name', 'inspection_shift', 'inspection_notes']
for col in cat_col:
    df[col] = df[col].astype(str)
    df[col] = label.fit_transform(df[col])
```

Feature Selection (x, y)

```
In [25]: x = df.drop(columns=['fabric_quality'])
y = df['fabric_quality']
```

Train-Test Split

The dataset is divided into 80% training data and 20% testing data.

```
In [26]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

```
In [ ]:
```

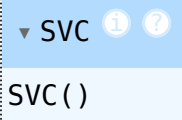
Feature Scaling

Support Vector Machines are sensitive to feature scales, therefore StandardScaler is applied to normalize the features.

```
In [27]: scaler=StandardScaler()  
x_train_scale = scaler.fit_transform(x_train)  
x_test_scale = scaler.transform(x_test)
```

Model Training

```
In [28]: svc = SVC()  
svc.fit(x_train_scale, y_train)
```

```
Out[28]:   
SVC()
```

Model Prediction

```
In [29]: y_pred = svc.predict(x_test_scale)
```

Model Evaluation

Accuracy Score

```
In [30]: acc = accuracy_score(y_test, y_pred)  
acc
```

```
Out[30]: 0.9285436893203883
```

Classification Report

```
In [31]: print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
High	0.94	0.95	0.94	1745
Low	0.94	0.94	0.94	1679
Medium	0.90	0.90	0.90	1726
accuracy			0.93	5150
macro avg	0.93	0.93	0.93	5150
weighted avg	0.93	0.93	0.93	5150

```
In [ ]:
```

SVM Kernel Comparison and Performance Evaluation

Different kernels create different decision boundaries. The RBF kernel usually performs well for nonlinear data.

```
In [60]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

kernels = ['linear', 'sigmoid', 'poly', 'rbf']

kernel_names = []
acc_list = []
prec_list = []
rec_list = []
f1_list = []

for kernel in kernels:
    model = SVC(kernel=kernel)
    model.fit(x_train_scale, y_train)

    y_pred = model.predict(x_test_scale)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='weighted')
    rec = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')

    kernel_names.append(kernel)
    acc_list.append(acc)
    prec_list.append(prec)
    rec_list.append(rec)
    f1_list.append(f1)
```

In []:

In [61]: **import** pandas **as** pd

```
svm_df = pd.DataFrame({
    "Kernel": kernel_names,
    "Accuracy": acc_list,
    "Precision": prec_list,
    "Recall": rec_list,
    "F1 Score": f1_list
})

svm_df = svm_df.sort_values(by="Accuracy", ascending=False)

display(svm_df)
```

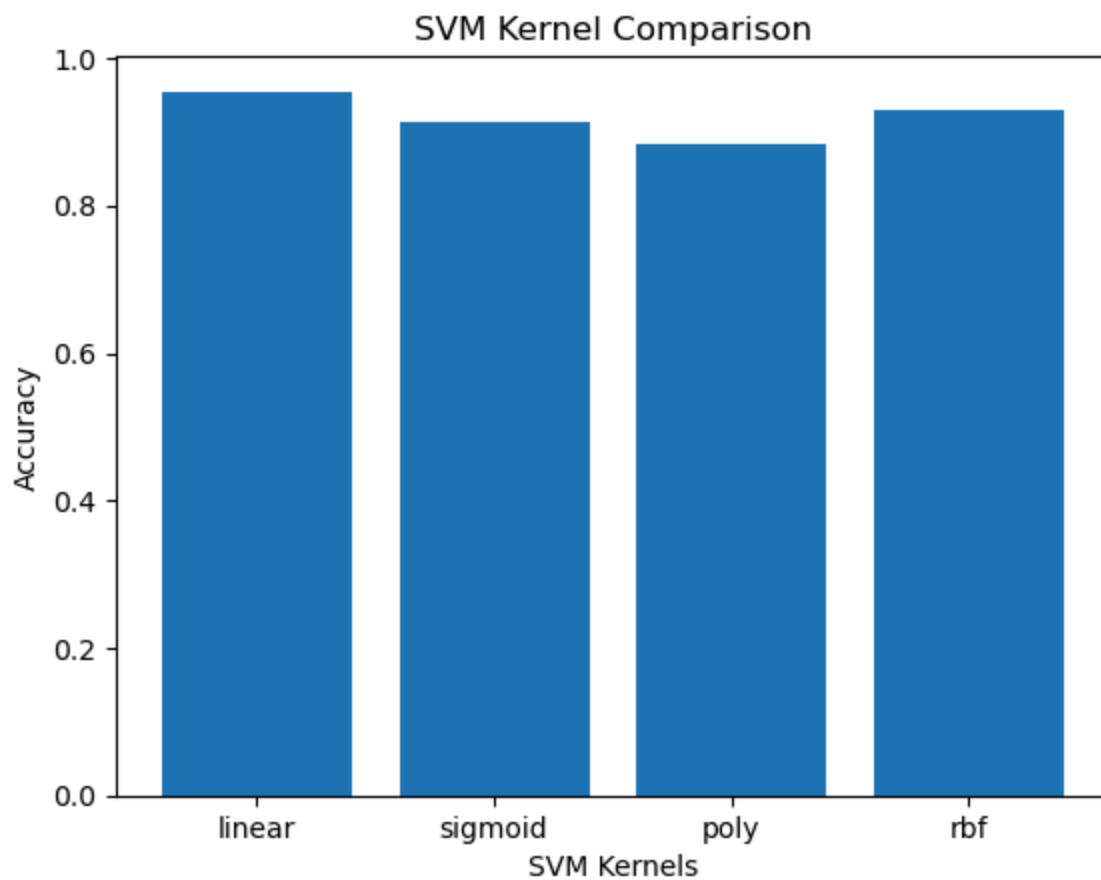
	Kernel	Accuracy	Precision	Recall	F1 Score
0	linear	0.954757	0.954711	0.954757	0.954713
3	rbf	0.928544	0.928439	0.928544	0.928482
1	sigmoid	0.913398	0.913259	0.913398	0.913303
2	poly	0.883495	0.883758	0.883495	0.883616

In [62]: **import** matplotlib.pyplot **as** plt

```
plt.figure()
plt.bar(kernel_names, acc_list)

plt.xlabel("SVM Kernels")
plt.ylabel("Accuracy")
plt.title("SVM Kernel Comparison")

plt.show()
```



In []:

In []:

Verifying Target Class Labels

```
In [33]: print(y_train.unique())  
         print(y_test.unique())
```

```
['Low' 'Medium' 'High']  
['Low' 'High' 'Medium']
```

```
In [34]: print("Accuracy:", accuracy_score(y_test, y_pred))  
         print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))  
         print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Accuracy: 0.9285436893203883

Confusion Matrix:

```
[[1651  13  81]
 [ 12 1584  83]
 [ 88  91 1547]]
```

Classification Report:

	precision	recall	f1-score	support
High	0.94	0.95	0.94	1745
Low	0.94	0.94	0.94	1679
Medium	0.90	0.90	0.90	1726
accuracy			0.93	5150
macro avg	0.93	0.93	0.93	5150
weighted avg	0.93	0.93	0.93	5150

Confusion Matrix

```
In [35]: cm = confusion_matrix(y_test, y_pred)
cm
```

```
Out[35]: array([[1651,  13,  81],
               [ 12, 1584,  83],
               [ 88,  91, 1547]])
```

Cross Validation Score

Cross-validation evaluates model stability by training and testing the model on multiple data splits.

```
In [36]: from sklearn.model_selection import cross_val_score
score = cross_val_score(svc,x_train_scale,y_train, cv=5)
print(score)
print(score.mean()*100)
```

```
[0.92257282 0.92402913 0.91941748 0.91917476 0.92427184]
92.18932038834951
```

```
In [37]: cv_scores = cross_val_score(svc,x_train_scale,y_train, cv=5, scoring='accuracy')
print("CV Scores:",cv_scores)
print("Mean CV Accuracy:", cv_scores.mean())
```

```
CV Scores: [0.92257282 0.92402913 0.91941748 0.91917476 0.92427184]
Mean CV Accuracy: 0.9218932038834952
```

The cross-validation scores indicate consistent model performance across different folds.

Multi-Class ROC Curve and AUC Analysis for SVM

The ROC (Receiver Operating Characteristic) curve is used to evaluate the performance of a classification model at different classification thresholds.

Since this problem involves multiple classes (High, Medium, Low fabric quality), a one-vs-rest approach is used to compute the ROC curve for each class individually.

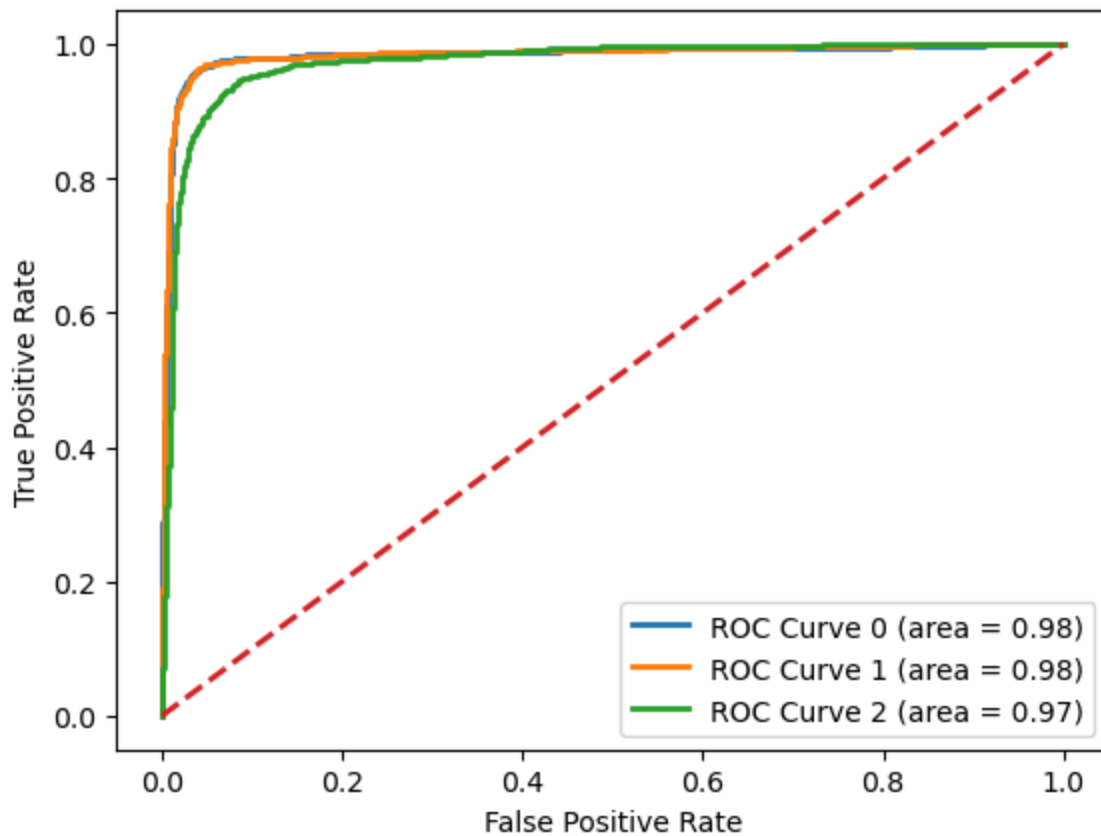
The Area Under the Curve (AUC) measures the model's ability to distinguish between classes. A higher AUC value indicates better classification performance.

```
In [38]: from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, roc_auc_score
from sklearn.model_selection import cross_val_score
```

```
In [39]: svc = SVC(probability=True)
svc.fit(x_train_scale, y_train)
if len(set(y_test))>1:
    y_test_binarize = label_binarize(y_test, classes=svc.classes_)
    n_classes = y_test_binarize.shape[1]
    y_score = svc.predict_proba(x_test_scale)

    fpr = dict()
    tpr = dict()
    roc_auc = dict()

    for i in range(n_classes):
        fpr[i], tpr[i], _ = roc_curve(y_test_binarize[:, i], y_score[:, i])
        roc_auc[i] = roc_auc_score(y_test_binarize[:, i], y_score[:, i])
    for i in range(n_classes):
        plt.plot(fpr[i], tpr[i], lw = 2, label='ROC Curve {0} (area = {1:0.2f})'.format(i, roc_auc[i]))
    plt.plot([0, 1], [0, 1], lw = 2, linestyle='--')
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.legend()
    plt.show()
```



High ROC values indicate strong classification capability.

GridSearchCV

GridSearchCV tests multiple combinations of hyperparameters to find the best K value and distance metric.

```
In [40]: param_grid = {  
          'C': [0.1, 1, 10]  
        }  
svc = SVC(kernel='linear')  
grid = GridSearchCV(svc,param_grid,cv=5,scoring='accuracy')  
grid.fit(x_train_scale, y_train)  
print("Best Parameters: ", grid.best_params_)  
print("Best CV Accuracy: ", grid.best_score_)  
best_grid_model = grid.best_estimator_
```

```
Best Parameters: {'C': 10}  
Best CV Accuracy: 0.9511650485436892
```


RandomizedSearchCV

RandomizedSearchCV is used to optimize the hyperparameters of the SVM model by randomly sampling combinations of parameters from a predefined distribution.

```
In [41]: param_dist = {
          'C': [0.1, 1, 10],
          'kernel': ['rbf'],
          'gamma': ['scale', 0.1, 0.01]
        }
        svc = SVC()
        random_search = RandomizedSearchCV(svc, param_distributions=param_dist, n_iter=5)
        random_search.fit(x_train_scale, y_train)
        print("Best Parameters: ", random_search.best_params_)
        print("Best CV Accuracy: ", random_search.best_score_)

        best_random_model = random_search.best_estimator_
```

```
Best Parameters: {'kernel': 'rbf', 'gamma': 'scale', 'C': 1}
Best CV Accuracy: 0.9182523372482138
```

```
In [ ]:
```

Comparison of Hyperparameter Tuning Methods (GridSearch vs RandomSearch)

```
In [42]: best_grid_model = grid.best_estimator_

        y_pred_grid = best_grid_model.predict(x_test_scale)
        print("Test Accuracy (GridSearch):",
              accuracy_score(y_test, y_pred_grid))
        print(classification_report(y_test, y_pred_grid))
```

```
Test Accuracy (GridSearch): 0.954757281553398
```

	precision	recall	f1-score	support
High	0.96	0.96	0.96	1745
Low	0.96	0.97	0.96	1679
Medium	0.95	0.93	0.94	1726
accuracy			0.95	5150
macro avg	0.95	0.95	0.95	5150
weighted avg	0.95	0.95	0.95	5150

```
In [43]: best_random_model = random_search.best_estimator_

        y_pred_random = best_random_model.predict(x_test_scale)
        print("Test Accuracy (RandomSearch):",
```

```
accuracy_score(y_test, y_pred_random))
print(classification_report(y_test, y_pred_random))
```

```
Test Accuracy (RandomSearch): 0.9285436893203883
      precision    recall  f1-score   support

   High       0.94       0.95       0.94       1745
    Low       0.94       0.94       0.94       1679
   Medium     0.90       0.90       0.90       1726

 accuracy
macro avg       0.93       0.93       0.93       5150
weighted avg     0.93       0.93       0.93       5150
```

In []:

In []:

Improving Model Performance with Advanced Algorithms

In []:

```
In [53]: from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import LabelEncoder

# ENCODE TARGET VARIABLE
le = LabelEncoder()
y_train = le.fit_transform(y_train)
y_test = le.transform(y_test)

# METRIC FUNCTION
def evaluate(y_true, y_pred):
    return (
        accuracy_score(y_true, y_pred),
        precision_score(y_true, y_pred, average='weighted'),
        recall_score(y_true, y_pred, average='weighted'),
        f1_score(y_true, y_pred, average='weighted')
    )

# MODELS

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier
```

```

from catboost import CatBoostClassifier
from lightgbm import LGBMClassifier

models = {
    "Logistic Regression": (
        LogisticRegression(max_iter=1000),
        {"C": [0.1, 1, 10]}
    ),

    "KNN": (
        KNeighborsClassifier(),
        {"n_neighbors": [3, 5, 7]}
    ),

    "Decision Tree": (
        DecisionTreeClassifier(),
        {"max_depth": [3, 5, 10]}
    ),

    "Random Forest": (
        RandomForestClassifier(),
        {"n_estimators": [100, 200], "max_depth": [None, 10]}
    ),

    "AdaBoost": (
        AdaBoostClassifier(),
        {"n_estimators": [50, 100]}
    ),

    "Gradient Boosting": (
        GradientBoostingClassifier(),
        {"n_estimators": [100, 200], "learning_rate": [0.05, 0.1]}
    ),

    "XGBoost": (
        XGBClassifier(use_label_encoder=False, eval_metric='mlogloss'),
        {"n_estimators": [100, 200], "learning_rate": [0.05, 0.1]}
    ),

    "CatBoost": (
        CatBoostClassifier(verbose=0),
        {"iterations": [100, 200], "learning_rate": [0.05, 0.1]}
    ),

    "LightGBM": (
        LGBMClassifier(),
        {"n_estimators": [100, 200], "learning_rate": [0.05, 0.1]}
    )
}

# TRAIN + GRID SEARCH
results = []

```

```

for name, (model, params) in models.items():

    grid = GridSearchCV(
        model,
        params,
        cv=2,
        scoring='accuracy',
        n_jobs=-1
    )

    grid.fit(x_train_scale, y_train)

    best_model = grid.best_estimator_

    y_pred = best_model.predict(x_test_scale)

    acc, prec, rec, f1 = evaluate(y_test, y_pred)

    results.append([
        name,
        acc,
        prec,
        rec,
        f1,
        grid.best_params_
    ])

# RESULTS TABLE

columns = ['Model', 'Accuracy', 'Precision', 'Recall', 'F1 Score', 'Best Param']

comparison_df = pd.DataFrame(results, columns=columns)

comparison_df = comparison_df.sort_values(by='Accuracy', ascending=False)
comparison_df

```

[LightGBM] [Info] Auto-choosing col-wise multi-threading, the overhead of testing was 0.001497 seconds.

You can set `force\_col\_wise=true` to remove the overhead.

[LightGBM] [Info] Total Bins 3100

[LightGBM] [Info] Number of data points in the train set: 20600, number of used features: 22

[LightGBM] [Info] Start training from score -1.080381

[LightGBM] [Info] Start training from score -1.098564

[LightGBM] [Info] Start training from score -1.117231

Out[53]:

	Model	Accuracy	Precision	Recall	F1 Score	Best Params
0	Logistic Regression	0.955146	0.955137	0.955146	0.955138	{'C': 0.1}
7	CatBoost	0.938447	0.938839	0.938447	0.938577	{'iterations': 200, 'learning_rate': 0.1}
8	LightGBM	0.923883	0.923975	0.923883	0.923925	{'learning_rate': 0.1, 'n_estimators': 200}
6	XGBoost	0.922718	0.923185	0.922718	0.922891	{'learning_rate': 0.1, 'n_estimators': 200}
5	Gradient Boosting	0.920000	0.920802	0.920000	0.920264	{'learning_rate': 0.1, 'n_estimators': 200}
3	Random Forest	0.892233	0.892180	0.892233	0.892199	{'max_depth': None, 'n_estimators': 200}
2	Decision Tree	0.833981	0.833589	0.833981	0.833734	{'max_depth': 10}
4	AdaBoost	0.791650	0.819519	0.791650	0.796434	{'n_estimators': 100}
1	KNN	0.684466	0.675703	0.684466	0.678314	{'n_neighbors': 7}

In []:

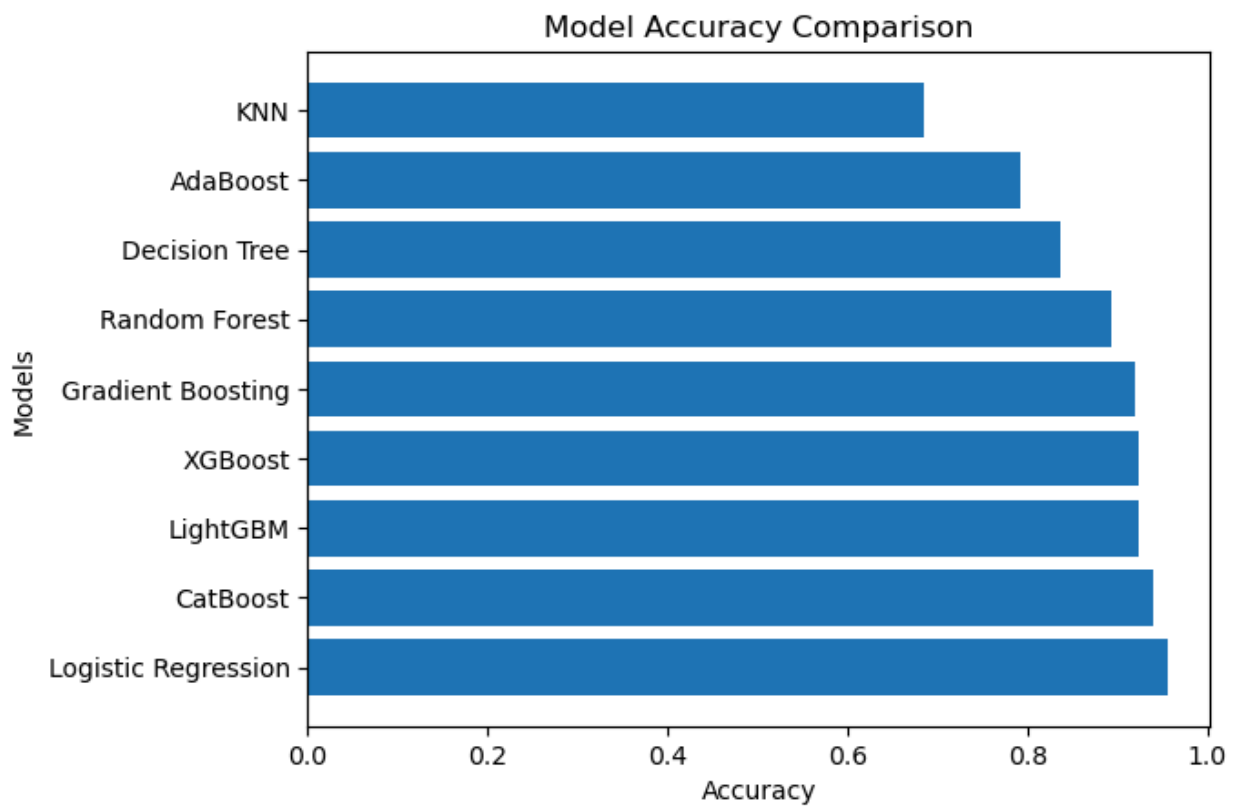
In [63]: **import** matplotlib.pyplot **as** plt

```
plt.figure()

plt.barh(df["Model"], df["Accuracy"])

plt.xlabel("Accuracy")
plt.ylabel("Models")
plt.title("Model Accuracy Comparison")

plt.show()
```



Logistic Regression performs comparably to top ensemble models, while boosting methods slightly lead overall, and KNN remains the lowest performer.

In []:

In []:

In []:

In []: